

Ai Assisted Coding

Name: G.Sreeja

Enroll.no:2503a52692

Batch:24

Experiment 6: AI-Based Code Completion: Working with suggestions for classes, loops, conditionals

Task1: Use an AI code completion tool to generate a loop-based program.

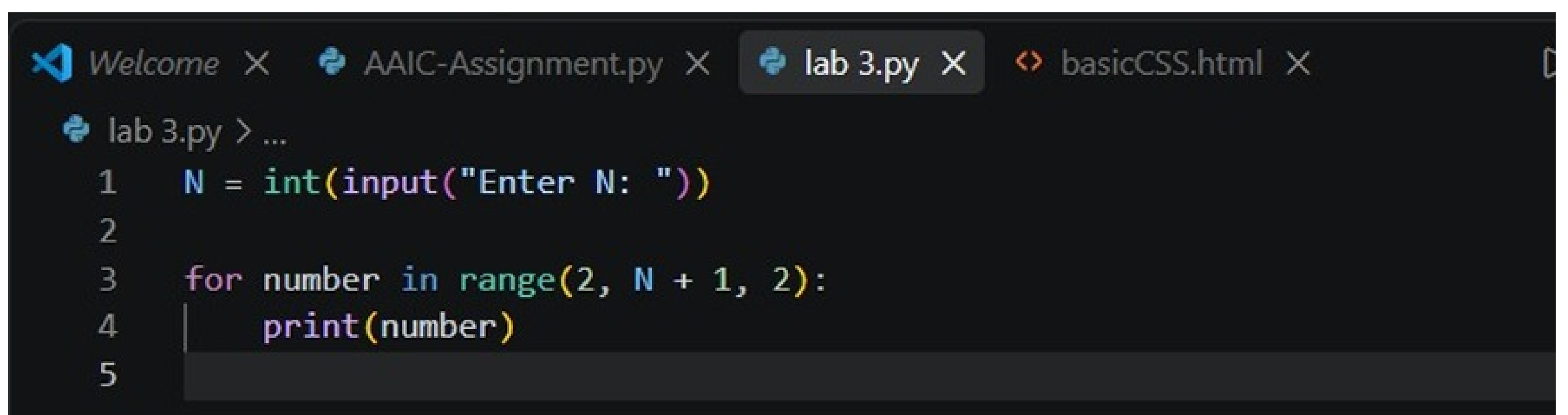
Prompt:

“Generate Python code to print all even numbers between 1 and N using a loop.”

Expected Output:

- AI-generated loop logic.
- Identification of loop type used (for or while).
- Validation with sample inputs.

Input

A screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'Welcome', 'Aaic-Assignment.py', 'lab 3.py' (which is the active tab), and 'basicCSS.html'. The code in the 'lab 3.py' tab is as follows:

```
lab 3.py > ...
1  N = int(input("Enter N: "))
2
3  for number in range(2, N + 1, 2):
4      print(number)
5
```

Output

```
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/ASUS/OneDrive/Desktop/AAIC-LAB/lab3.py"
Enter N: 10
2
4
6
8
10
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/ASUS/OneDrive/Desktop/AAIC-LAB/lab3.py"
Enter N: 25
2
4
6
8
10
12
14
16
18
20
22
24
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB>
```

Task2: Use an AI code completion tool to combine loops and conditionals.

Prompt:

“Generate Python code to count how many numbers in a list are even and odd.”

Expected Output:

- AI-generated code using loop and if condition.
- Correct count validation.
- Explanation of logic flow.

Input

```
lab 3.py 7 ...
1  numbers = list(map(int, input("Enter numbers separated by spaces: ").split()))
2
3  even_count = sum(number % 2 == 0 for number in numbers)
4  odd_count = sum(number % 2 != 0 for number in numbers)
5
6  print("Even numbers:", even_count)
7  print("Odd numbers:", odd_count)
8  |
```

Output

```
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/ASUS/OneDrive/Desktop/AAIC-LAB/lab 3.py"
Enter numbers separated by spaces: 10 15 22 33 48 51 60 7
Even numbers: 4
Odd numbers: 4
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & C:\Users\ASUS\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/ASUS/OneDrive/Desktop/AAIC-LAB/lab 3.py"
Enter numbers separated by spaces: 1 14 10 96 02 45
Even numbers: 4
Odd numbers: 2
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> █
```

Task3: Use an AI tool to complete a Python class that validates user input.

Prompt:

“Generate a Python class User that validates age and email using conditional statements.”

Expected Output:

- AI-generated class with validation logic.
- Verification of condition handling.
- Test cases for valid and invalid inputs.

Input

```
lab3.py
1  class User:
2      def __init__(self, name, age, email):
3          self.name = name
4          self.age = age
5          self.email = email
6          self.is_valid = self.validate()
7
8      def validate(self):
9          if self.age < 0 or self.age > 120:
10             print("Invalid age: age must be between 0 and 120")
11             return False
12          if "@" not in self.email or "." not in self.email:
13             print("Invalid email: must contain '@' and '.'")
14             return False
15          print(f"{self.name} is valid. Age: {self.age}, Email: {self.email}")
16          return True
17
18
19 name = input("Enter name: ")
20 age = int(input("Enter age: "))
21 email = input("Enter email: ")
22 user = User(name, age, email)
```

Output

```
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & python lab3.py
Enter name: Vinay
Enter age: 25
Enter email: vinay@gmail.com
Vinay is valid. Age: 25, Email: vinay@gmail.com
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & python lab3.py
Enter name: Sam
Enter age: 150
Enter email: sam@gmail.com
Invalid age: age must be between 0 and 120
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & python lab3.py
Enter name: Anu
Enter age: 30
Enter email: anugmail.com
Invalid email: must contain '@' and '.'
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB>
```

Task4: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Expected Output:

- AI-generated class code.
- Verification of correctness and completeness of class structure.
- Minor manual improvements (if needed) with justification.

Input

```

1  class Student:
2      def __init__(self, name, roll_number, marks):
3          self.name = name
4          self.roll_number = roll_number
5          self.marks = marks
6
7      def total_marks(self):
8          return sum(self.marks)
9
10     def average_marks(self):
11         return self.total_marks() / len(self.marks)
12
13
14     name = input("Enter student name: ")
15     roll_number = input("Enter roll number: ")
16     marks = list(map(int, input("Enter marks separated by spaces: ").split()))
17
18     student = Student(name, roll_number, marks)
19     print(f"Total marks: {student.total_marks()}")
20     print(f"Average marks: {student.average_marks():.2f}")

```

Output

```

PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & python lab3.py
Enter student name: Rahul
Enter roll number: 2503A52710
Enter marks separated by spaces: 85 90 78 92 88
Total marks: 433
Average marks: 86.60
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB>

```

Task5: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

“Generate a Python program for a simple bank account system using class, loops, and conditional statements.”

Expected Output:

- Complete AI-generated program.
- Identification of strengths and limitations of AI suggestions.
- Reflection on how AI assisted coding productivity.



Input

```
1  class BankAccount:
2      def __init__(self, owner, balance=0):
3          self.owner = owner
4          self.balance = balance
5
6      def deposit(self, amount):
7          if amount <= 0:
8              print("Deposit amount must be positive.")
9          else:
10             self.balance += amount
11             print(f"Deposited {amount}. New balance: {self.balance}")
12
13     def withdraw(self, amount):
14         if amount <= 0:
15             print("Withdrawal amount must be positive.")
16         elif amount > self.balance:
17             print("Insufficient balance.")
18         else:
19             self.balance -= amount
20             print(f"Withdrew {amount}. New balance: {self.balance}")
21
22     def check_balance(self):
23         print(f"Current balance: {self.balance}")
24
25
26 name = input("Enter account holder name: ")
27 account = BankAccount(name)
28
29 while True:
30     print("1. Deposit  2. Withdraw  3. Check Balance  4. Exit")
31     choice = input("Enter choice: ")
32     if choice == "1":
33         amount = float(input("Enter amount to deposit: "))
34         account.deposit(amount)
35     elif choice == "2":
36         amount = float(input("Enter amount to withdraw: "))
37         account.withdraw(amount)
38     elif choice == "3":
39         account.check_balance()
40     elif choice == "4":
41         print("Exiting... Thank you!")
42         break
43     else:
44         print("Invalid choice. Please try again.")
```

Output

```
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB> & python lab3.py
Enter account holder name: Priya
1. Deposit  2. Withdraw  3. Check Balance  4. Exit
Enter choice: 1
Enter amount to deposit: 5000
Deposited 5000.0. New balance: 5000.0
1. Deposit  2. Withdraw  3. Check Balance  4. Exit
Enter choice: 3
Current balance: 5000.0
1. Deposit  2. Withdraw  3. Check Balance  4. Exit
Enter choice: 2
Enter amount to withdraw: 7000
Insufficient balance.
1. Deposit  2. Withdraw  3. Check Balance  4. Exit
Enter choice: 2
Enter amount to withdraw: 2000
Withdrew 2000.0. New balance: 3000.0
1. Deposit  2. Withdraw  3. Check Balance  4. Exit
Enter choice: 4
Exiting... Thank you!
PS C:\Users\ASUS\OneDrive\Desktop\AAIC-LAB>
```